

**Group 12: AI Declaration, Link to Steps File**

| Name                     | ID number |
|--------------------------|-----------|
| Van Asher Alcantara      | 12340898  |
| Enrique Miguel M. Aragon | 12227811  |
| Angelica Therese Clavano | 12206245  |
| Job Lozada               | 12307246  |

Steps: [See the Steps.xlsx file](#)  Steps

**Declaration of AI usage: Van Alcantara**

I used ChatGPT for help in formulating a regex needed for cleaning data.

Extent of use: Minimal, only for grammar checking and searching

- Prompt use: "Give me the list of characters used to include all letters used in any language and also include the special characters that are standard and found in most Bible translations"
- ChatGPT reply: Sure!  
Here are the characters that will be allowed:

**Letters:**

A–Z, a–z, and extended Latin letters (é, ü, ñ, etc.)

**Numbers:**

0–9

**Whitespace:**

Spaces, tabs, newlines

**Punctuation:**

Period ., comma ,, exclamation !, question ?, colon :, semicolon ;, single ' and double quotes ", curly quotes “ ” ‘ ’, hyphen –, parentheses ( )

If you wanted to express this as a regex pattern for filtering text, it might look something like:

```
[^A-Za-zÀ-ÖØ-öø-ÿ\u00C0-\u024F\u1E00-\u1EFF0-9\s\.\,\!\?:\;\'\\"-]
```

- Prompt use: Can you explain what each part does?  
[^A-Za-zÀ-ÖØ-öø-ÿ\u00C0-\u024F\u1E00-\u1EFF0-9\s\.\,\!\?:\;\'\\"-]
- ChatGPT reply:

♦ Overall Structure

- `[...]` — defines a **character class**, which matches *any one character* inside the brackets.
- `^` (at the start) — *negates* the class.
  - Meaning: instead of matching what's inside, it matches **any character not listed**.
  - So this regex matches **unwanted characters**.

---

#### ♦ Inside the brackets `[...]`

| Part                            | Meaning                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>A-Za-z</code>             | All <b>basic English letters</b> (uppercase and lowercase).                                             |
| <code>À-Öö-öø-<br/>-ÿ</code>    | <b>Extended Latin letters</b> (common accented characters like é, ü, ñ, etc.).                          |
| <code>\u00C0-<br/>\u024F</code> | Unicode range for <b>Latin Extended-A and B</b> — adds more accented letters and special Latin symbols. |
| <code>\u1E00-<br/>\u1EFF</code> | Unicode range for <b>Latin Extended Additional</b> — covers rare accented forms used in some languages. |
| <code>0-9</code>                | All <b>digits</b> (0 through 9).                                                                        |
| <code>\s</code>                 | Any <b>whitespace</b> character (spaces, tabs, newlines).                                               |
| <code>\.</code>                 | A literal <b>period</b> .                                                                               |
| <code>\,</code>                 | A literal <b>comma</b> .                                                                                |
| <code>\!</code>                 | A literal <b>exclamation mark</b> .                                                                     |
| <code>\?</code>                 | A literal <b>question mark</b> .                                                                        |
| <code>\:</code>                 | A literal <b>colon</b> .                                                                                |
| <code>\;</code>                 | A literal <b>semicolon</b> .                                                                            |
| <code>\' "</code>               | Both <b>single</b> and <b>double quotes</b> .                                                           |
| <code>" " " "</code>            | <b>Curly quotes</b> (typographic quotation marks).                                                      |
| <code>\-</code>                 | A <b>hyphen</b> or <b>dash</b> .                                                                        |
| <code>\(\)</code>               | <b>Parentheses</b> .                                                                                    |

Using ChatGPT helped in learning because it can give a step by step understanding instead of just giving answers directly. It can explain certain concepts especially if they can be very technical.

### Declaration of AI usage: Enrique Aragon

I used Deepseek in order to aid with debugging a few RegEx statements when the VSCode text editor was not working well. I also used it occasionally to ask what parts of the text were still unclear.

Extent of use: Minimal, only for debugging when faced with VSCode display issues and checking cleanliness of the text

- Prompt use: San Mateo 1 1 Libro de  
Are these tab charas and not spaces?
- Deepseek reply: Yes, those are **tabs** between "San Mateo 1", "1", and "Libro".
- Prompt use: Check if I've cleaned it enough with regex so that its pure sentences.
- Deepseek reply: Based on a quick scan, the text still contains verse numbers and references (e.g., 6'Ika, Betlehem..., 9-Kaya naghale sinda., (Gibo. Kan maaraman...), which should be removed for "pure sentences."

Using Deepseek helped in learning because VSCode's text editor led me to believe my RegEx was wrong, when it wasn't. It makes the process of debugging less of a hassle, which is especially helpful when working with such a large corpora.

### Declaration of AI usage: Jack Clavano

I used ChatGPT (website) 5, ChatGPT-5 mini (Github Copilot within VSCode)

**Extent of use:** Moderate – used for inspiration, then reworted significantly.

Prompts below, text size was reduced to fit more.

ChatGPT 5:

- **Prompt use:** "What does this error mean? ----- ParserError Traceback (most recent call last) Cell In[19], line 17 15 if cleaned\_file\_path.exists() and cleaned\_file\_path.stat().st\_size > 0: 16 with cleaned\_file\_path.open("r", encoding="utf-8") as f: --> 17 df = pd.read\_csv(cleaned\_file\_path, sep='|', on\_bad\_lines='error') 18 excel = df.to\_excel(pd\_writer, sheet\_name=f'{lang}-verses', index=False) # false indexing bc we already have it 19 print(f"Added {lang} to verses.xlsx") File c:\Users\gicac\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\io\parsers\readers.py:1026, in read\_csv(filepath\_or\_buffer, sep, delimiter, header, names, index\_col, usecols, dtype, engine, converters, true\_values, false\_values, skipinitialspace, skiprows, skipfooter, nrows, na\_values, keep\_default\_na, na\_filter, verbose, skip\_blank\_lines, parse\_dates, infer\_datetime\_format, keep\_date\_col, date\_parser, date\_format, dayfirst, cache\_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding\_errors, dialect, on\_bad\_lines, delim\_whitespace, low\_memory, memory\_map, float\_precision, storage\_options, dtype\_backend) 1013 kwds\_defaults = \_refine\_defaults\_read( 1014 dialect, 1015 delimiter, (...) 1022 dtype\_backend=dtype\_backend, 1023 ) 1024 kwds.update(kwds\_defaults) -> 1026 return \_read(filepath\_or\_buffer, kwds) File c:\Users\gicac\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\io\parsers\readers.py:626, in \_read(filepath\_or\_buffer, kwds) 623 return parser 625 with parser: --> 626 return parser.read(nrows) File c:\Users\gicac\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\io\parsers\readers.py:1923, in

```

TextFileReader.read(self, nrows) 1916 nrows = validate_integer("nrows", nrows) 1917 try: 1918 # error: "ParserBase" has
no attribute "read" 1919 ( 1920 index, 1921 columns, 1922 col_dict, -> 1923 ) = self._engine.read( # type:
ignore[attr-defined] 1924 nrows 1925 ) 1926 except Exception: 1927 self.close() File
c:\Users\gicac\AppData\Local\Programs\Python\Python312\Lib\site-packages\pandas\io\parsers\c_parser_wrapper.py:234
, in CParserWrapper.read(self, nrows) 232 try: 233 if self.low_memory: --> 234 chunks =
self._reader.read_low_memory(nrows) 235 # destructive to chunks 236 data = _concatenate_chunks(chunks) File
pandas/_libs/parsers.pyx:838, in pandas._libs.parsers.TextReader.read_low_memory() File pandas/_libs/parsers.pyx:905,
in pandas._libs.parsers.TextReader._read_rows() File pandas/_libs/parsers.pyx:874, in
pandas._libs.parsers.TextReader._tokenize_rows() File pandas/_libs/parsers.pyx:891, in
pandas._libs.parsers.TextReader._check_tokenize_status() File pandas/_libs/parsers.pyx:2061, in
pandas._libs.parsers.raise_parser_error() ParserError: Error tokenizing data. C error: Expected 4 fields in line 92, saw 5"

```

- **ChatGPT 5 reply:** This error is raised by `pandas` when reading a CSV file because the number of fields (columns) in a line does not match what it expects based on earlier lines. (explains key part of the error, why it happens, how to fix: 1. `**Inspect the problematic line**`, 2. `**Handle bad lines gracefully**`, 3. `**Fix the source file**`) \* Open the CSV and manually correct line 92 (make sure it has exactly 4 columns). \* If extra `|` is inside text data, ensure it is quoted.  `**Summary**`: Pandas expected 4 columns, but line 92 had 5. This means your CSV has a malformed row or extra delimiter. Inspect line 92 and either fix the data or configure `'on_bad_lines'` to handle it automatically. Would you like me to suggest code that automatically `**detects and reports all problematic lines**` instead of stopping at the first one?
- **Prompt use:** "Given the cleaned verse file, how come the next verse is combined in one when I import the txt to excel or sheets?" (attached file: masbateno-cleaned.txt)

**ChatGPT 5 reply:** The issue you're running into when importing the cleaned verse text file into Excel or Google Sheets is due to `**line breaks and delimiters**`. From looking at your file : \* Each verse appears as a single line, but some verses continue onto the next line without a clear delimiter (e.g., Mateo 1:5 continues with Mateo 1:6 on the same line). \* Excel/Sheets expects `**consistent delimiters**` (like commas, tabs, or pipes `|`) to separate columns and line breaks to separate rows. \* If a verse spans multiple lines without proper handling, Excel/Sheets will treat it as a single cell value and merge verses into one row. `### Why verses are combined` \* Some lines contain two verses because of how line breaks were handled when cleaning the file. \* Example: `*Mateo|1|5*` ends with text, and `*Mateo|1|6*` follows immediately on the same line instead of starting on a new line. That means when Excel reads it, it sees one row containing both verses. `### How to fix it` 1. `**Check delimiter consistency**` (code here) 2. `**Normalize line breaks**` 3. `**Use a cleaning script**` (Python or regex) (code here) Then import the new file into Excel/Sheets. - - -  In short: the verses are combined because multiple verse markers are on the same line. You'll need to re-split them (via regex or manual cleaning) so every `'Book|Chapter|Verse|Text'` is on its own row. Would you like me to generate a `**fully cleaned version of your file**` where each verse is guaranteed to be on its own row, ready for Excel?

- **Prompt use:** "using regex, how can i search for the third | per line"
- **ChatGPT 5 reply:** `^(?:[^\|]*){2}[^\|]*K\|`

ChatGPT-5 mini (mode within Github Copilot):

- **Prompt use:** "in this code block, which one handles 12, 13" (attached file: main.ipynb Cell 12:30-86)
- **ChatGPT-5 mini reply:** "The line that handles "12,13" is:  
# ...existing code...  
text = re.sub(r'(\d+,\d+)', r'\1|', text, flags = re.MULTILINE)  
# ...existing code..."
- **Prompt use:** "how do i run a python file that exists in the same directory in a single cell on an ipynb notebook"
- **ChatGPT-5 mini reply:** "Use one of these in a single notebook cell (assumes the .py file is in the same folder as the notebook): (code for quick run `%run script.py`, Run with the same Python interpreter (prints stdout/stderr): `!py script.py`, Import as a module, etc.)
- **Prompt use:** "given the code here, how do i make sure it will SKIP if the bibleAbbreviation or bibleNumber contains "SKIP""
- **ChatGPT-5 mini reply:** Use a case-insensitive substring check for "SKIP" (and include lang too). Replace the existing SKIP check in the for loop with this: (code)
- **Prompt use:** "can pandas turn txt files into a single excel with multiple pages"
- **ChatGPT-5 mini reply:** Yes. Use `pandas.read_csv` to load each .txt and `pandas.ExcelWriter` to write multiple sheets. If files have different schemas, normalize columns before concatenation. You need an Excel writer engine (openpyxl or xlsxwriter). Install via pip.

Using ChatGPT 5/ChatGPT-5 mini/Github Copilot helped in learning because sometimes searching up the problem won't give you similar or exact solutions to a specific problem you're going through. In my experience, it also gives you the words or terms (like libraries) that you're looking for.

## Declaration of AI usage: Job Lozada

I used Claude 4.5 Sonnet.

Extent of Use: To help familiarize myself with working with the Pandas library's functionality for converting Dataframes into .csv files.

- **Prompt use:** How can I use pandas functions in order to transform the text in this cleaned text into a .csv that has columns for the Bible Book, Chapter, Verse Number, and verse contents?
- **Claude 4.5 Sonnet Reply:** When you call `df.to_csv()`, here's what happens under the hood: **\*\*1.** The DataFrame is iterated row by row **\*\*** Pandas goes through your DataFrame sequentially, one row at a time. Each row contains values from all the columns (Book, Chapter, Verse #, Text, etc.). **\*\*2.** Each row is converted into a CSV line **\*\*** For each row, pandas takes the values and joins them together with a delimiter (comma by default). So a row like:  
...
  - `{'Book': 'Matthew', 'Chapter': 1, 'Verse #': 1, 'Text': 'Some text here'}`
  - ...
  - Becomes this CSV line:  
...
  - `Matthew,1,1,Some text here`
  - **\*\*3.** The header row is written first (if `header=True`) **\*\*** Before any data rows, pandas writes the column names as the first line:  
...
  - `Book,Chapter,Verse #,Text`
  - **\*\*4.** Data is written to a file **\*\*** All those CSV lines (header + data rows) are written sequentially to a file on your hard drive. This happens fairly efficiently even with large datasets because pandas doesn't load everything into memory at once when writing—it streams it out. **\*\*5.** Special handling for problematic data **\*\*** Pandas handles edge cases:- **\*\*Commas in text\*\***: If your text contains a comma, pandas wraps the entire field in quotes: `"text, with comma"`- **\*\*Quotes in text\*\***: It escapes them by doubling: `"text with ""quotes"""`- **\*\*Missing values\*\***: Converts `'NaN'` or `'None'` to empty cells (or whatever you specify with `'na_rep'` parameter)- **\*\*Index column\*\***: If `'index=True'` (default), it adds row numbers as the first column **\*\*6.** Encoding is applied **\*\*** The `'encoding='utf-8'` parameter tells pandas how to convert characters into bytes. This is crucial for your Bible text because it contains various languages and special characters. UTF-8 can handle basically any character from any language. **\*\*So for your Bible data specifically:\*\*** Your DataFrame might look like this in memory:  
...

|   | Book    | Chapter | Verse # | Text                           |
|---|---------|---------|---------|--------------------------------|
| 0 | Matthew | 1       | 1       | An account of the genealogy... |
| 1 | Matthew | 1       | 2       | Abraham was the father of...   |
| 2 | Matthew | 1       | 3       | and Judah the father of...     |

  
...**\*\*When you call `df.to_csv('bible_verses.csv', index=False, encoding='utf-8')`, pandas creates a file that contains:**  
...
    - `Book,Chapter,Verse #,Text`
    - `Matthew,1,1,An account of the genealogy...`
    - `Matthew,1,2,Abraham was the father of...`
    - `Matthew,1,3,and Judah the father of...`
    - **\*\*The `'index=False'` prevents row numbers from being added. The `'encoding='utf-8'` ensures that any special characters in the text are properly saved in a format that can be read by other programs. **\*\*Why this matters for your use case:\*\*** Since you're working with Bible translations in different languages, UTF-8 encoding ensures that Greek letters, accents, non-Latin scripts, etc., all get saved correctly. If you used the wrong encoding (like ASCII), those characters would be corrupted or lost.**